BICOL UNIVERSITY - POLANGUI
Web Systems and Technology

ALSOR CO.

BS INFORMATION TECHNOLOGY

PROPONENTS:

Ivan Jacob Milan
Nadel Volante
James Espinosa
Erick John Bonaobra
Manuel Angelo Loma

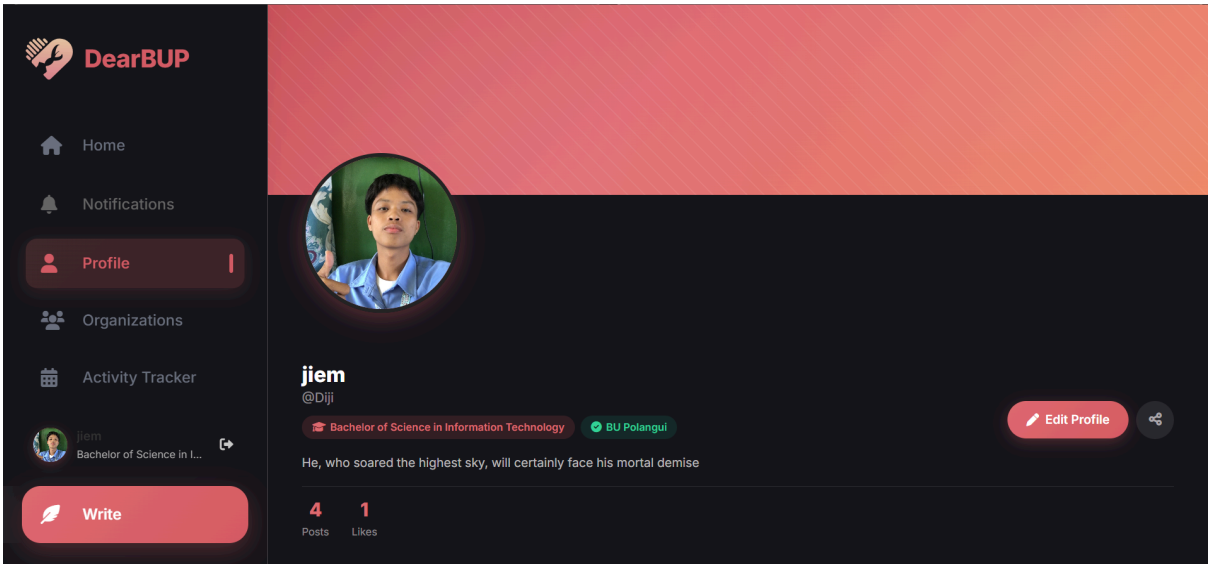
DearBUP
Feel. Write. Share.

Week 15: Phase 6 Update and Delete Functionalities

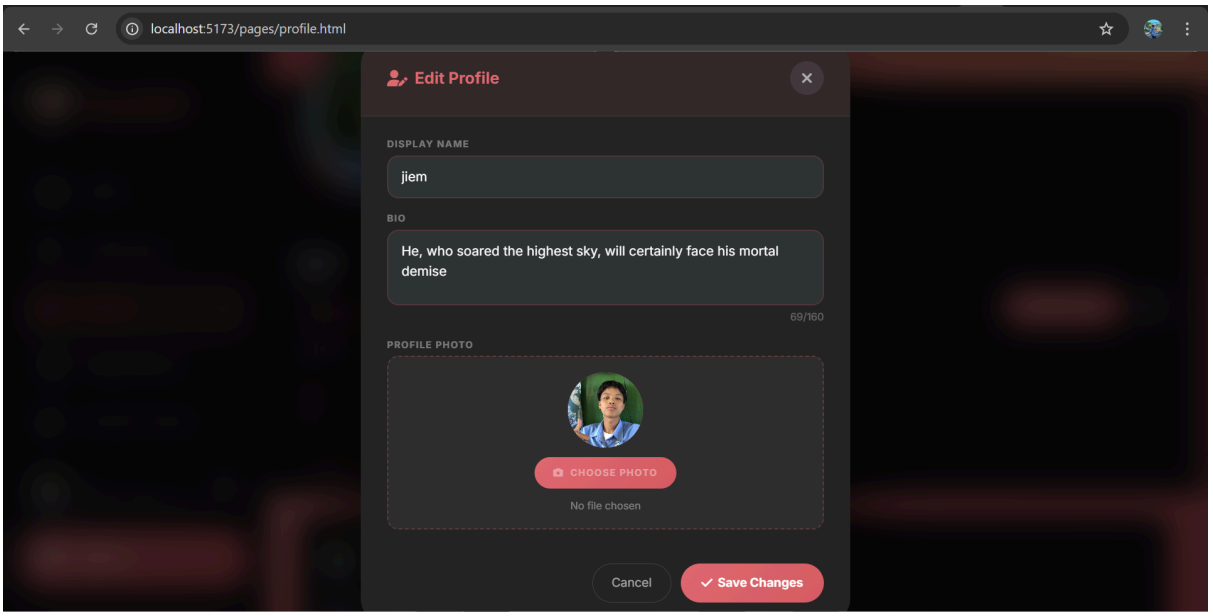
EDIT MODAL/PAGE

Edit/Update details for profiles

If you click the profile menu on the left there is a desired page wherein you can view your account and at the same time personalize it. There is a button on the right that let you edit your details.



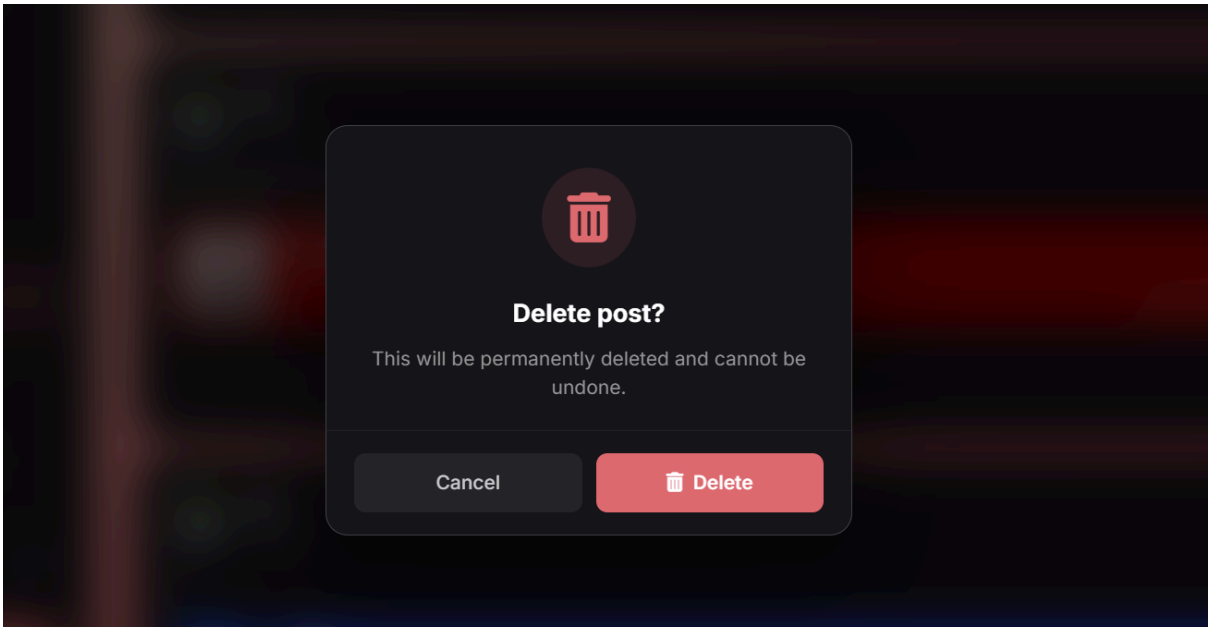
After clicking the edit profile you are now directed to an edit modal page where you can manage your details and edit it. The available function for now is the function where you can edit your display name, your bio, and profile picture. If you click the save changes button, your profile will be updated on the website and it is synced on the database.



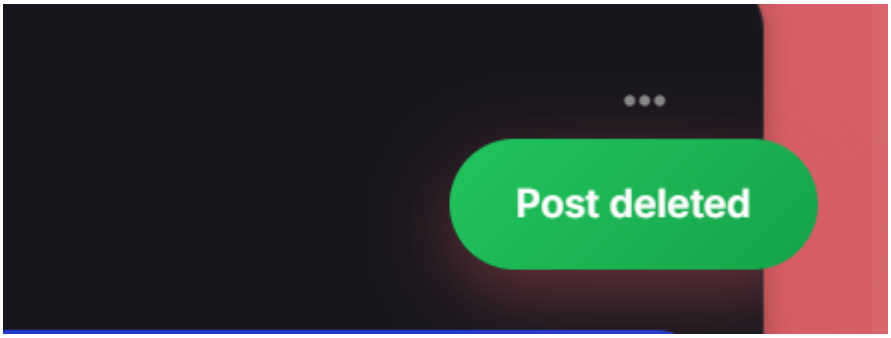
Edit and Delete function for Posts

A three dot design in the post consists of two functions where you can edit or delete the post. This function only works if the post you've been deleting is yours and you have access to the post.

If you click the delete button or the trash icon, a modal popup will show as a second reminder for you to decide if you want to delete it or not. This popup modal helps the user to decide carefully and it avoids error in decisions.



Clicking the Delete button will automatically set the small notification or confirmation for the deletion of posts. This also works on the letter post where you can delete the post. The only differences is the edit function in the post is not available in the letter story.

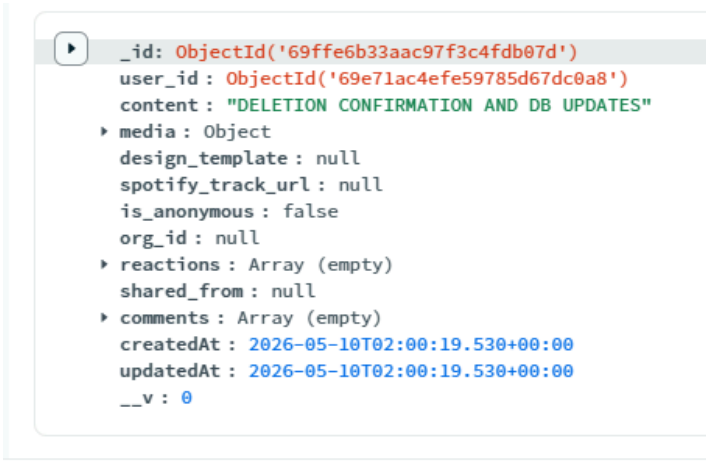


DELETION CONFIRMATION AND DB UPDATES

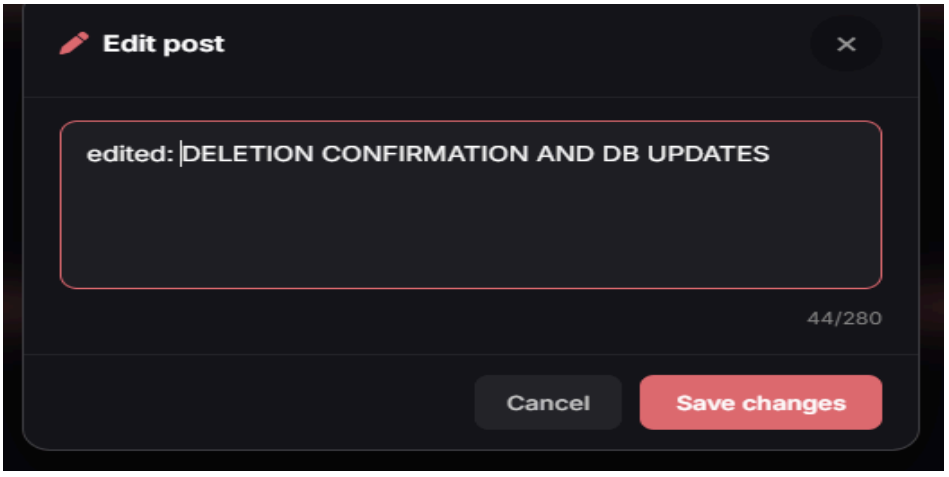
A. UPDATE

Post update for localhost

Post update for Database



Edited Post in localhost

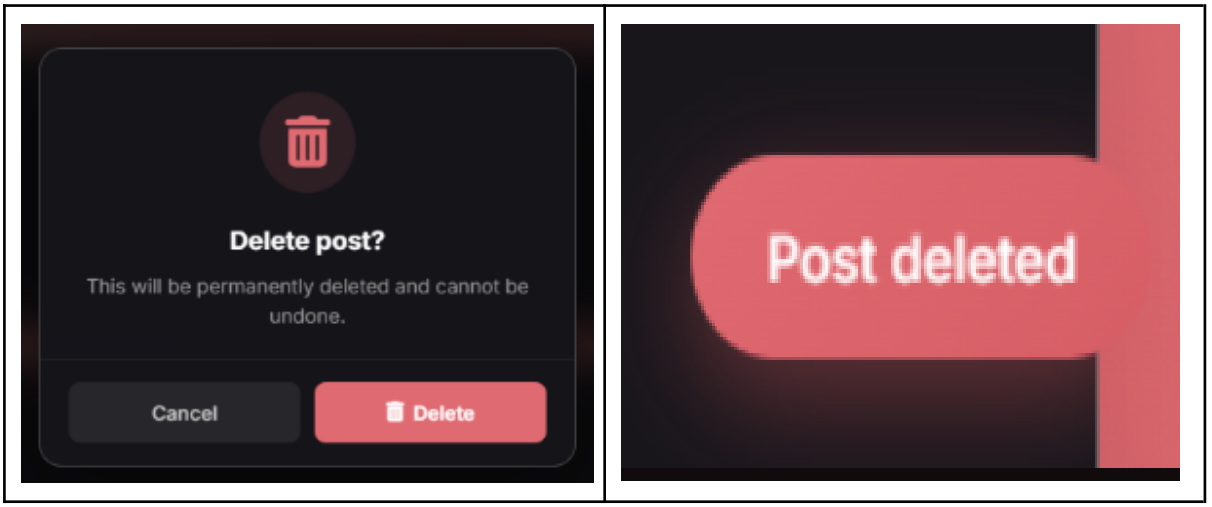


Edited Post in Database

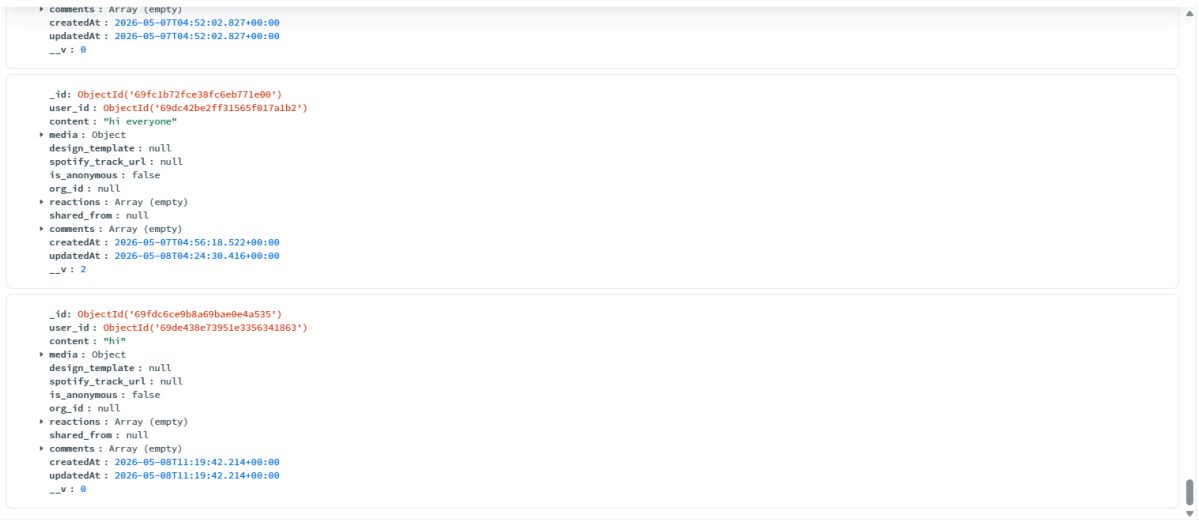

```
_id: ObjectId('69ffe6b33aac97f3c4fdb07d')
user_id : ObjectId('69e71ac4efe59785d67dc0a8')
content : "edited: DELETION CONFIRMATION AND DB UPDATES"
▸ media : Object
design_template : null
spotify_track_url : null
is_anonymous : false
org_id : null
▸ reactions : Array (empty)
shared_from : null
▸ comments : Array (empty)
createdAt : 2026-05-10T02:00:19.530+00:00
updatedAt : 2026-05-10T02:01:55.297+00:00
__v : 0
```


B. DELETION

Delete a post in localhost



Disappearance of the data in database



Since the post has been deleted, the database removed it from the lists of available posts or the existing ones. There is no trash in MongoDb atlas so the post is just deleted and removed.

JS CODE SNIPPET FOR EDIT & DELETE FUNCTION

- **Post Edit and Delete**

```
// ---------------- UPDATE ----------------
export const updatePost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.user_id.toString() !== req.user.id)
      return res.status(403).json({ message: "Unauthorized" });

    Object.assign(post, req.body);
    await post.save();
    res.json(post);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- DELETE ----------------
export const deletePost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.user_id.toString() !== req.user.id)
      return res.status(403).json({ message: "Unauthorized" });

    await post.deleteOne();
    res.json({ message: "Deleted" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

export const getPostById = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id)
      .populate("user_id", "username display_name avatar_url")
      .populate("comments.user_id", "username display_name avatar_url")
      .populate({
        path: "shared_from",
        populate: {
          path: "user_id",
          select: "username display_name avatar_url"
        }
      });
```

```
    if (!post) return res.status(404).json({ message: "Post not found"
});
    res.json(post);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

These functions handle backend resource management by providing secure endpoints to modify or remove content from the database. Both the update and delete operations include a critical authorization check to ensure the requesting user owns the post before executing changes via Mongoose methods like `Object.assign()` or `.deleteOne()`. Additionally, the retrieval logic uses `.populate()` to stitch together related user and comment data, ensuring the frontend receives a complete, nested data object for rendering.

BACKEND ROUTE AND CONTROLLER CODE SNIPPETS

A. ROUTES

- **Authentication Route**

```
import express from "express";
import { register, login, sendOtp } from
"../controllers/authController.js";

const router = express.Router();


router.post("/send-otp", sendOtp);
router.post("/register", register);
router.post("/login", login);

export default router;
```

This code defines an Express router that manages user authentication by mapping three specific POST endpoints to their corresponding logic: it provides a route to send a verification code via `send-otp`, a route to create new user accounts via `register`, and a route to verify credentials via `login`. By centralizing these endpoints in a dedicated router file, the application keeps its identity management logic organized and secure, ensuring that sensitive data is handled in the request body rather than the URL.

- **CBO officers route**

```
import express from "express";
import {
  assignOfficer,
  getOrgOfficers,
} from "../controllers/cboOfficerController.js";

import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

router.post("/assign", authMiddleware, assignOfficer);
router.get("/org/:orgId", getOrgOfficers);

export default router;
```

This code defines a specialized router for managing **organization officers**, featuring a protected **POST** route to `/assign` that uses `authMiddleware` to ensure only authorized users can appoint someone to a role. It also includes a public (or differently scoped) **GET** route that uses a URL parameter `(:orgId)` to fetch and display a list of officers belonging to a specific organization. Effectively, it acts as the gateway for linking users to official positions within the system's organizational hierarchy.

- **CBO Routes**

```
// routes/cboRoutes.js
import express from "express";
import {
  getOrganizations,
  getOrganizationById,
  joinOrganization,
  leaveOrganization,
} from "../controllers/cboController.js";
import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

// Public
router.get("/",    getOrganizations);
router.get("/:id", getOrganizationById);

// Protected — need to be logged in
router.post("/:id/join",  authMiddleware, joinOrganization);
router.post("/:id/leave", authMiddleware, leaveOrganization);

export default router;
```

This code establishes a Community-Based Organization (CBO) router that separates public information from member-specific actions by using a mix of HTTP methods and security layers. The first two GET routes allow any user to browse a general list of organizations or view specific details for a single ID, while the POST routes for joining or leaving an organization are shielded by authMiddleware to ensure only authenticated users can modify their membership status. By leveraging dynamic path parameters (/:id), the router efficiently directs requests to the controller logic responsible for fetching data or updating the database based on the user's login state.

- **Event Routes**

```
import { Router } from 'express';
import { body } from 'express-validator';
import {
  getEvents,
  getEventById,
  createEvent,
  updateEvent,
  deleteEvent,
  toggleRSVP,
  getEventStats,
} from '../controllers/eventController.js';
import { protect, authorize } from '../middleware/authMiddleware.js';
```

```
const router = Router();
const eventValidation = [
            body('title').trim().notEmpty().withMessage('Title       is
required.').isLength({ max: 150 }),
       body('content').trim().notEmpty().withMessage('Description    is
required.').isLength({ max: 1000 }),
    body('date').isISO8601().withMessage('Valid  date  (ISO  8601)  is
required.'),
  body('org_id').isMongoId().withMessage('Valid org_id is required.'),
];
// Public (read) - auth optional so is_going is populated when logged
in
router.get('/',      getEvents);
router.get('/stats', getEventStats);
router.get('/:id',   getEventById);
// Protected
router.post('/',
  protect, authorize('cbo_officer', 'admin'),
  eventValidation,
  createEvent
);
router.put('/:id',
  protect, authorize('cbo_officer', 'admin'),
  updateEvent
);
router.delete('/:id',
  protect, authorize('admin'),
  deleteEvent
);
// Any logged-in student can RSVP
router.post('/:id/rsvp', protect, toggleRSVP);
export default router;
```

This script defines a comprehensive Event Management router that integrates data validation, multi-level authorization, and public access. It provides open GET routes for browsing events and statistics, while strictly protecting administrative actions—such as creating, updating, or deleting events—by requiring specific roles like `cbo_officer` or `admin` via the `authorize` middleware and validating input data through `eventValidation`. Additionally, it includes a student-focused POST route for RSVPing, ensuring that while anyone can view events, only authenticated users with the correct permissions can modify event details or interact with the attendance system.

- Feedback Routes

```
import express from "express";
import { submitFeedback, getDeptFeedback } from
"../controllers/feedbackController.js";
import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

// Only logged-in users can submit feedback
router.post("/", authMiddleware, submitFeedback);

// Get feedback for a specific department
router.get("/dept/:deptId", authMiddleware, getDeptFeedback);

export default router;
```

This script establishes a **Feedback router** that secures all interactions behind an authMiddleware, ensuring that only authenticated users can access the feedback system. It provides a **POST** route for users to submit their input and a **GET** route to retrieve feedback filtered by a specific department ID, maintaining a private channel for organizational communication. By requiring a login for both submitting and viewing data, the router ensures accountability and prevents unauthorized access to internal department evaluations.

- **Letter Routes**

```
import express from "express";
import {
  createLetter,
  getAllLetters,
  getLetterById,
  deleteLetter
} from "../controllers/letterController.js";
// Change "protect" to "authMiddleware" here
import { authMiddleware } from "../middleware/authMiddleware.js";
const router = express.Router();
router.get("/", getAllLetters);
router.get("/:id", getLetterById);

// Use the correct name here too
router.post("/", authMiddleware, createLetter);
router.delete("/:id", authMiddleware, deleteLetter);
export default router;
```

This script defines a **Letter Management router** that provides public access to view all letters or a specific letter via **GET** routes, while restricting creation and deletion to authenticated users through the authMiddleware. It serves as a controlled interface for handling correspondence, ensuring that sensitive actions like posting new content or removing existing files are protected from unauthorized access.

- **Notification Route**

```
import express from "express";
import {
  getNotifications,
  markAsRead,
  getUnreadCount,
} from "../controllers/notificationController.js";

import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

router.get("/", authMiddleware, getNotifications);
router.get("/count", authMiddleware, getUnreadCount);
router.put("/:id/read", authMiddleware, markAsRead);

export default router;
```

This script defines a Notification router that secures all endpoints behind authMiddleware, ensuring that only logged-in users can interact with their personal alerts. It provides GET routes to retrieve a full notification list or a simple unread count, alongside a PUT route to update specific notifications as read based on their ID.

- **Organization Route**

```
import express from "express";
import {
  createOrganization,
  getOrganizations,
  getOrganizationById,
  joinOrganization,
  approveMember,
  leaveOrganization
} from "../controllers/organizationController.js";

import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

// CRUD
router.post("/", authMiddleware, createOrganization);
router.get("/", getOrganizations);
router.get("/:id", getOrganizationById);

// membership
router.post("/:id/join", authMiddleware, joinOrganization);
```

```
router.post("/:orgId/approve/:userId", authMiddleware, approveMember);
router.post("/:id/leave", authMiddleware, leaveOrganization);

export default router;
```

This script defines an **Organization router** that manages the lifecycle and membership of groups, using authMiddleware to protect administrative and personal actions like creating an organization or joining one. It provides public **GET** routes for browsing organizations, while utilizing specialized **POST** routes with dynamic URL parameters to handle complex logic such as approving a specific user's membership within a specific organization.

- **Post Route**

```
import express from "express";
import multer from "multer";

import {
  createPost,
  getPosts,
  getPostById,
  reactToPost,
  addComment,
  sharePost,
  updatePost,
  deletePost,
  searchSpotify
} from "../controllers/postsController.js";

import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();
const upload = multer({ dest: "uploads/" });

router.post("/", authMiddleware, upload.single("media"), createPost);
router.get("/", getPosts);

router.get("/spotify/search", searchSpotify);
router.get("/:id", authMiddleware, getPostById);

router.post("/:id/react", authMiddleware, reactToPost);
router.post("/:id/comment", authMiddleware, addComment);
router.post("/:id/share", authMiddleware, sharePost);

router.put("/:id", authMiddleware, updatePost);
router.delete("/:id", authMiddleware, deletePost);

export default router;
```

This script establishes a Social Post router that manages content creation and interaction, notably integrating Multer middleware to handle file uploads for media-rich posts. While it offers public access to the general feed, it protects specific post views, Spotify searches, and engagement actions like reacting, commenting, or sharing behind authMiddleware to ensure a personalized and secure user experience.

- **Profile Route**

```
import express from "express";
import {
  getProfile,
  updateProfile,
  getUserProfileById,
  getUserPosts
} from "../controllers/profileController.js";
import { authMiddleware } from "../middleware/authMiddleware.js";

const router = express.Router();

router.get("/", authMiddleware, getProfile);
router.put("/", authMiddleware, updateProfile);
router.get("/user/:id", authMiddleware, getUserProfileById);
router.get("/user/:id/posts", authMiddleware, getUserPosts);

// VIEW OTHER USER PROFILE
router.get("/user/:id", authMiddleware, getUserProfileById);

export default router;
```

This script establishes a User Profile router that restricts all access to the authMiddleware, ensuring that only logged-in users can view or modify personal information and activity. It provides routes for users to manage their own settings via GET and PUT methods, while also allowing them to fetch public profile details and post histories for other users through dynamic ID-based endpoints.

- **Upload Route**

```
import express from "express";
import { authMiddleware } from "../middleware/authMiddleware.js";
import { uploadAvatar, uploadPostMedia } from "../controllers/uploadController.js";

const router = express.Router();

router.post("/avatar", authMiddleware, uploadAvatar);
router.post("/post-media", authMiddleware, uploadPostMedia);

export default router;
```

This script defines a File Upload router that provides secure endpoints for handling media assets, specifically for user avatars and post-related content. By wrapping both POST routes in authMiddleware, the system ensures that only authenticated users can upload files to the server's storage or cloud provider.

B. CONTROLLERS

- **Auth Controller**

```
import User from "../models/UserModel.js";
import Otp from "../models/OtpModel.js";
import bcrypt from "bcryptjs";
import jwt from "jsonwebtoken";
import { sendOtpEmail } from "../utils/sendEmail.js";

//  SEND OTP
export const sendOtp = async (req, res) => {
  try {
    const { email } = req.body;

    if (!email)
      return res.status(400).json({ message: "Email is required" });

    // 1. CALL THE CHECK (and make it case-insensitive/trimmed)
    const isBUEmail = (email) => email.trim().toLowerCase().endsWith("@bicol-u.edu.ph");

    // 2. IF IT'S NOT A BU EMAIL, STOP HERE
    if (!isBUEmail(email)) {
      return res.status(403).json({
          message: "Access restricted. Only @bicol-u.edu.ph emails are allowed."
      });
    }
```

```
    // generate 6-digit OTP
            const   code   =   Math.floor(100000   +   Math.random()   *
900000).toString();

    // delete existing OTP for same email
    await Otp.deleteMany({ email });

    // save new OTP (expires in 5 minutes)
    await Otp.create({
      email,
      code,
      expiresAt: new Date(Date.now() + 5 * 60 * 1000),
    });

    // Only reaches here if the isBUEmail check passed
    await sendOtpEmail(email, code);

    res.json({ message: "OTP sent successfully" });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- REGISTER (WITH OTP CHECK) ----------------
export const register = async (req, res) => {
  try {
    const {
      email,
      username,
      password,
      display_name,
      user_type,
      student_id,
      course,
      otp
    } = req.body;

    // REQUIRED FIELDS
     if (!email || !username || !password || !user_type || !student_id
|| !course || !otp) {
       return res.status(400).json({ message: "All fields including OTP
are required" });
    }
```

```
    // 🔥 VERIFY OTP FIRST
    const existingOtp = await Otp.findOne({ email });

    if (!existingOtp)
      return res.status(400).json({ message: "No OTP found. Please request again." });

    if (existingOtp.code !== otp)
      return res.status(400).json({ message: "Invalid OTP" });

    if (existingOtp.expiresAt < new Date())
      return res.status(400).json({ message: "OTP expired" });

    // DELETE OTP AFTER SUCCESS
    await Otp.deleteMany({ email });

    // CHECK DUPLICATES
    const emailExists = await User.findOne({ email });
    if (emailExists)
      return res.status(400).json({ message: "Email already exists" });

    const usernameExists = await User.findOne({ username });
    if (usernameExists)
      return res.status(400).json({ message: "Username already taken" });

    const studentIdExists = await User.findOne({ student_id });
    if (studentIdExists)
      return res.status(400).json({ message: "Student ID already registered" });

    // ROLE VALIDATION
    const allowedRoles = ["student", "officer", "department_head", "admin"];
    if (!allowedRoles.includes(user_type))
      return res.status(400).json({ message: "Invalid user role" });

    // HASH PASSWORD
    const hashedPassword = await bcrypt.hash(password, 10);

    // CREATE USER
    const user = await User.create({
      email,
```

```
      username,
      password: hashedPassword,
      user_type,
      student_id,
      course,
      display_name: display_name || username,
    });

    const { password: _, ...safeUser } = user.toObject();

    res.status(201).json({
      message: "User registered successfully",
      user: safeUser,
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- LOGIN ----------------
export const login = async (req, res) => {
  try {
    const { email, password } = req.body;

    if (!email || !password)
          return  res.status(400).json({  message:  "Email  and  password
required" });

    const user = await User.findOne({ email });
    if (!user)
      return res.status(404).json({ message: "User not found" });

    const match = await bcrypt.compare(password, user.password);
    if (!match)
      return res.status(401).json({ message: "Invalid password" });

    const token = jwt.sign(
      { id: user._id, role: user.user_type },
      process.env.JWT_SECRET,
      { expiresIn: "7d" }
    );

    const { password: _, ...safeUser } = user.toObject();
```

```
    res.json({
      message: "Login successful",
      token,
      user: safeUser,
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages a secure, university-exclusive authentication system by strictly validating Bicol University email addresses and utilizing a two-step verification process. It handles the generation and expiration of 6-digit OTPs for email verification, enforces a registration flow that checks for duplicate student records, and manages user sessions through hashed passwords and JWT token generation.

- **CBO Controller**

```
// controllers/cboController.js
import CBO from "../models/CboModel.js";

// GET /api/organization
export const getOrganizations = async (req, res) => {
  try {
    const { department, search } = req.query;
    const filter = {};

    if (department && department !== "all") {
      filter.department = department;
    }

    if (search) {
      filter.$or = [
        { org_name:    { $regex: search, $options: "i" } },
        { acronym:     { $regex: search, $options: "i" } },
        { description: { $regex: search, $options: "i" } },
      ];
    }

    const orgs = await CBO.find(filter)
      .populate("created_by", "username display_name")
      .populate("members.user_id", "username display_name")
      .sort({ createdAt: -1 });

    res.json(orgs);
```

```
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// GET /api/organization/:id
export const getOrganizationById = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id)
      .populate("created_by", "username display_name")
      .populate("members.user_id", "username display_name");

     if (!org) return res.status(404).json({ message: "Organization not
found" });

    res.json(org);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// POST /api/organization/:id/join
export const joinOrganization = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id);
     if (!org) return res.status(404).json({ message: "Organization not
found" });

      const userId = req.user.id; // comes from authMiddleware → jwt
decoded

    const existing = org.members.find(
      (m) => m.user_id?.toString() === userId
    );

    if (existing) {
      const msg =
        existing.status === "approved"
          ? "You are already a member."
          : "Your request is already pending.";
      return res.status(400).json({ message: msg });
    }
```

```
        org.members.push({ user_id: userId, role: "member", status:
"pending" });
    await org.save();

    const updated = await CBO.findById(req.params.id)
      .populate("created_by", "username display_name")
      .populate("members.user_id", "username display_name");

      res.json({ message: "Join request sent! Waiting for approval.",
org: updated });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// POST /api/organization/:id/leave
export const leaveOrganization = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id);
     if (!org) return res.status(404).json({ message: "Organization not
found" });

    const userId = req.user.id;
    const before = org.members.length;

    org.members = org.members.filter(
      (m) => m.user_id?.toString() !== userId
    );

    if (org.members.length === before) {
        return res.status(400).json({ message: "You are not a member of
this organization." });
    }

    await org.save();
    res.json({ message: "You have left the organization." });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages Community-Based Organization (CBO) data and membership by providing features to filter, search, and dynamically update organization rosters. It allows users to browse organizations based on department or keywords, while strictly managing the join and leave lifecycle by tracking member statuses and preventing duplicate requests.

- **CBO Officer Controller**

```
import CBOOfficer from "../models/CboOfficerModel.js";
import CBO from "../models/CboModel.js";
import User from "../models/UserModel.js";

// ---------------- ASSIGN OFFICER ROLE ----------------
// ONLY admin can assign officers
export const assignOfficer = async (req, res) => {
  try {
    const { user_id, org_id, officer_role } = req.body;

    const admin = await User.findById(req.user.id);

    if (!admin || admin.user_type !== "admin") {
        return res.status(403).json({ message: "Only admin can assign
officers" });
    }

    const org = await CBO.findById(org_id);
    const user = await User.findById(user_id);

    if (!org || !user) {
       return res.status(404).json({ message: "User or Organization not
found" });
    }

    // prevent duplicate officer
    const existing = await CBOOfficer.findOne({ user_id, org_id });

    if (existing) {
        return res.status(400).json({ message: "User already an officer
in this org" });
    }

    const officer = await CBOOfficer.create({
      user_id,
      org_id,
      officer_role, // "president", "vice_president", etc
      display_name: user.display_name || user.username,
    });

    res.status(201).json(officer);
  } catch (error) {
    res.status(500).json({ error: error.message });
```

```
  }
};

// ---------------- GET OFFICERS OF ORG ----------------
export const getOrgOfficers = async (req, res) => {
  try {
    const officers = await CBOOfficer.find({
      org_id: req.params.orgId,
    }).populate("user_id", "username display_name");

    res.json(officers);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages the appointment and retrieval of organization leadership by linking users to specific roles within a CBO. It strictly enforces an administrative check to ensure only authorized admins can assign roles like president or vice president, while also providing a public endpoint to list all officers currently serving a specific organization.

- Event Controller

```
import { validationResult } from 'express-validator';
import Event from '../models/EventModel.js';

// ── Helpers ──────────────────────────────────────────
const buildFilter = (query) => {
  const filter = { is_cancelled: false };

  const now = new Date();
  const todayStart = new Date(now); todayStart.setHours(0, 0, 0, 0);
  const todayEnd   = new Date(now); todayEnd.setHours(23, 59, 59, 999);
  const weekEnd    = new Date(todayStart.getTime() + 7 * 24 * 3600 * 1000);

  // Status filter
  if (query.status === 'upcoming') filter.date = { $gt: todayEnd };
  else if (query.status === 'today') filter.date = { $gte: todayStart, $lte: todayEnd };
  else if (query.status === 'past')  filter.date = { $lt: todayStart };
  else if (query.status === 'soon')  filter.date = { $gte: todayStart, $lte: weekEnd };

  // Org filter
  if (query.org_id) filter.org_id = query.org_id;

  // Event type
  if (query.event_type) filter.event_type = query.event_type;

  // Text search (title / content)
  if (query.search) {
    filter.$or = [
      { title:   { $regex: query.search, $options: 'i' } },
      { content: { $regex: query.search, $options: 'i' } },
      { venue:   { $regex: query.search, $options: 'i' } },
```

```
    ];
  }

  return filter;
};

// ── GET /api/events ──────────────────────────────────────────
// Chronological list with filter / search / sort / pagination
export const getEvents = async (req, res, next) => {
  try {
    const filter = buildFilter(req.query);

    // sort: asc = oldest first (default for upcoming), desc = newest first
    const sortDir = req.query.sort === 'desc' ? -1 : 1;

    const page  = Math.max(1, parseInt(req.query.page)  || 1);
    const limit = Math.min(50, parseInt(req.query.limit) || 20);
    const skip  = (page - 1) * limit;

    const [events, total] = await Promise.all([
      Event.find(filter)
        .populate('org_id', 'org_name acronym logo_url color')
        .sort({ date: sortDir, createdAt: sortDir })
        .skip(skip)
        .limit(limit),
      Event.countDocuments(filter),
    ]);

    // Attach whether the requesting user has RSVP'd
    const userId = req.user?._id?.toString();
    const data = events.map((ev) => {
      const obj = ev.toJSON();
      obj.is_going = userId ? ev.rsvp_users.some((id) => id.toString() === userId) : false;
      return obj;
    });

    res.json({
      success: true,
      total,
      page,
      pages: Math.ceil(total / limit),
      data,
    });
  } catch (err) {
    next(err);
  }
};

// ── GET /api/events/:id ──────────────────────────────────────
export const getEventById = async (req, res, next) => {
  try {
    const event = await Event.findById(req.params.id)
      .populate('org_id', 'org_name acronym logo_url color description');
```

```
    if (!event) return res.status(404).json({ success: false, message: 'Event not found.' });

    const obj = event.toJSON();
    const userId = req.user?._id?.toString();
    obj.is_going = userId ? event.rsvp_users.some((id) => id.toString() === userId) : false;

    res.json({ success: true, data: obj });
  } catch (err) {
    next(err);
  }
};

// ── POST /api/events ──────────────────────────────────────
// CBO officers and admins only
export const createEvent = async (req, res, next) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(422).json({ success: false, errors: errors.array() });
    }

    const { org_id, title, content, date, time, venue, event_type, banner_url } = req.body;

    const event = await Event.create({ org_id, title, content, date, time, venue, event_type, banner_url });
    await event.populate('org_id', 'org_name acronym logo_url color');

    res.status(201).json({ success: true, data: event });
  } catch (err) {
    next(err);
  }
};

// ── PUT /api/events/:id ───────────────────────────────────
export const updateEvent = async (req, res, next) => {
  try {
    const errors = validationResult(req);
    if (!errors.isEmpty()) {
      return res.status(422).json({ success: false, errors: errors.array() });
    }

    const allowed = ['title', 'content', 'date', 'time', 'venue', 'event_type', 'banner_url', 'is_cancelled'];
    const updates = {};
    allowed.forEach((k) => { if (req.body[k] !== undefined) updates[k] = req.body[k]; });

    const event = await Event.findByIdAndUpdate(req.params.id, updates, {
      new: true, runValidators: true,
    }).populate('org_id', 'org_name acronym logo_url color');

    if (!event) return res.status(404).json({ success: false, message: 'Event not found.' });
```

```
    res.json({ success: true, data: event });
  } catch (err) {
    next(err);
  }
};

// ── DELETE /api/events/:id ──────────────────────────────
export const deleteEvent = async (req, res, next) => {
  try {
    const event = await Event.findByIdAndDelete(req.params.id);
    if (!event) return res.status(404).json({ success: false, message: 'Event not found.' });
    res.json({ success: true, message: 'Event deleted.' });
  } catch (err) {
    next(err);
  }
};

// ── POST /api/events/:id/rsvp ──────────────────────────
// Toggle RSVP for the authenticated user
export const toggleRSVP = async (req, res, next) => {
  try {
    const event = await Event.findById(req.params.id);
    if (!event) return res.status(404).json({ success: false, message: 'Event not found.' });

    if (event.is_cancelled) {
      return res.status(400).json({ success: false, message: 'Cannot RSVP to a cancelled event.' });
    }

    const now = new Date();
    if (new Date(event.date) < now) {
      return res.status(400).json({ success: false, message: 'Cannot RSVP to a past event.' });
    }

    const userId = req.user._id;
    const idx = event.rsvp_users.findIndex((id) => id.toString() === userId.toString());

    let going;
    if (idx === -1) {
      event.rsvp_users.push(userId);
      going = true;
    } else {
      event.rsvp_users.splice(idx, 1);
      going = false;
    }

    await event.save();

    res.json({
      success: true,
      going,
      rsvp_count: event.rsvp_users.length,
```

```
      message: going ? "You're going!" : 'RSVP removed.',
    });
  } catch (err) {
    next(err);
  }
};

// ── GET /api/events/stats ──────────────────────────────
// Summary counts used by the stats strip in the UI
export const getEventStats = async (req, res, next) => {
  try {
    const now = new Date();
    const todayStart = new Date(now); todayStart.setHours(0, 0, 0, 0);
    const todayEnd   = new Date(now); todayEnd.setHours(23, 59, 59, 999);

    const [total, upcoming, today, past, going] = await Promise.all([
      Event.countDocuments({ is_cancelled: false }),
      Event.countDocuments({ is_cancelled: false, date: { $gt: todayEnd } }),
       Event.countDocuments({ is_cancelled: false, date: { $gte: todayStart, $lte: todayEnd
} }),
      Event.countDocuments({ is_cancelled: false, date: { $lt: todayStart } }),
      req.user
        ? Event.countDocuments({ is_cancelled: false, rsvp_users: req.user._id })
        : Promise.resolve(0),
    ]);

    res.json({ success: true, data: { total, upcoming: upcoming + today, today, past, going
} });
  } catch (err) {
    next(err);
  }
};
```

This controller acts as the **central management hub for the event system**, handling everything from sophisticated data filtering to interactive user participation like RSVPs. It features a robust query builder that dynamically sorts and filters events by status (upcoming, today, or past), organization, and text search while managing pagination for performance. Additionally, it implements strict business logic for administrative tasks (create, update, delete) and real-time statistics generation, ensuring users always see accurate counts of relevant community activities.

- **Feedback Controller**

```
import Feedback from "../models/FeedbackModel.js";
```

```
import User from "../models/UserModel.js";
import Department from "../models/DepartmentModel.js";


// @desc    Submit new feedback to a department
// @route   POST /api/feedback
export const submitFeedback = async (req, res) => {
  try {
    const { dept_id, content, is_anonymous } = req.body;

    // 1. Validation
    if (!dept_id || !content) {
       return res.status(400).json({ message: "Department ID and content
are required" });
    }

    // 2. Check if department exists
    const department = await Department.findById(dept_id);
    if (!department) {
      return res.status(404).json({ message: "Department not found" });
    }

    // 3. Create feedback
    const feedback = await Feedback.create({
      user_id: req.user.id, // From authMiddleware
      dept_id,
      content,
      is_anonymous: is_anonymous || false,
    });

    res.status(201).json({
      message: "Feedback submitted successfully",
      feedback,
    });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// @desc        Get feedback for a specific department (For Dept
Heads/Admin)
// @route   GET /api/feedback/dept/:deptId
export const getDeptFeedback = async (req, res) => {
  try {
```

```
    // We populate user_id but only if it's NOT anonymous
       // However, usually Dept Heads shouldn't see names if it's
anonymous
            const feedbackList = await Feedback.find({ dept_id:
req.params.deptId })
      .populate("user_id", "display_name username") // Optional
      .sort({ createdAt: -1 });

     // Clean data: If anonymous, mask the user info before sending to
frontend
    const sanitizedFeedback = feedbackList.map(item => {
      const doc = item.toObject();
      if (doc.is_anonymous) {
        delete doc.user_id;
        doc.user_display_name = "Anonymous Student";
      } else {
        doc.user_display_name = doc.user_id?.display_name || "User";
      }
      return doc;
    });

    res.status(200).json(sanitizedFeedback);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages the secure submission and retrieval of student feedback by serving as a bridge between users and university departments. It validates incoming submissions to ensure department IDs exist and properly stores content while respecting the user's choice of anonymity. When retrieving data, the logic implements a "privacy filter" that dynamically sanitizes the response, masking sensitive user details for anonymous entries while providing full attribution for public ones, ensuring ethical data handling for department heads and administrators.

- **Letter Controller**

```
import Letter from "../models/LetterModel.js";
```

```
import User from "../models/UserModel.js";

// @desc    Create a new letter (Story-style)
// @route   POST /api/letters
export const createLetter = async (req, res) => {
  try {
    const {
      letter_title,
      letter_content,
      letter_type,
      is_anonymous,
      spotifySongName,
      spotifyArtist,
      spotifyTrackUri,
      spotifyImageUrl
    } = req.body;

    // 1. Basic Validation
    if (!letter_title || !letter_content) {
        return res.status(400).json({ message: "Title and content are
required" });
    }

    // 2. Find the user to get their default display name
    const user = await User.findById(req.user.id);
    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

    // 3. Logic for Anonymity
    // If is_anonymous is true, we set name to "Anonymous",
    // otherwise we use their chosen display_name or username
    const finalDisplayName = is_anonymous
      ? "Anonymous"
      : (user.display_name || user.username);

    // 4. Create the Letter
    const newLetter = await Letter.create({
      user_id: req.user.id,
      letter_title,
      letter_content,
      letter_type: letter_type || "general",
      display_name: finalDisplayName,
      is_anonymous,
```

```
      spotifySongName,
      spotifyArtist,
      spotifyTrackUri,
      spotifyImageUrl
    });

    res.status(201).json({
      message: "Letter posted successfully! 📩",
      letter: newLetter
    });

  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// @desc    Get all letters (The "Feed")
// @route   GET /api/letters
export const getAllLetters = async (req, res) => {
  try {
    const letters = await Letter.find()
      .populate("user_id", "username display_name avatar_url")
      .sort({ createdAt: -1 });
    res.status(200).json(letters);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// @desc    Get a single letter detail
// @route   GET /api/letters/:id
export const getLetterById = async (req, res) => {
  try {
    const letter = await Letter.findById(req.params.id)
        .populate("user_id", "username display_name avatar_url"); // ←
add this
      if (!letter) return res.status(404).json({ message: "Letter not
found" });
    res.status(200).json(letter);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
// @desc    Delete a letter (Only by the owner)
```

```
// @route   DELETE /api/letters/:id
export const deleteLetter = async (req, res) => {
  try {
    const letter = await Letter.findById(req.params.id);

    if (!letter) {
      return res.status(404).json({ message: "Letter not found" });
    }

    // Check if the user is the owner
    if (letter.user_id.toString() !== req.user.id) {
        return res.status(401).json({ message: "Unauthorized to delete
this letter" });
    }

    await letter.deleteOne();
    res.json({ message: "Letter removed" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller governs the **creation and management of community "letters,"** integrating multimedia elements like Spotify tracks with flexible identity settings. It processes incoming content by validating requirements and applying anonymity logic to determine the displayed author name before saving the entry to the database. Additionally, it provides endpoints for fetching a chronological feed or specific details using `.populate()` for user metadata, while strictly enforcing ownership permissions for any deletion requests.

- **Notification Controller**

```
import Notification from "../models/NotificationModel.js";

// GET USER NOTIFICATIONS
export const getNotifications = async (req, res) => {
  try {
    const notifications = await Notification.find({
      user_id: req.user.id,
    })
      .populate("from_user", "username display_name avatar_url")
      .sort({ createdAt: -1 });

    res.json(notifications);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

```
// MARK AS READ
export const markAsRead = async (req, res) => {
  try {
    await Notification.findByIdAndUpdate(req.params.id, {
      is_read: true,
    });

    res.json({ message: "Marked as read" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
//Notification Indicator
export const getUnreadCount = async (req, res) => {
  try {
    const unreadCount = await Notification.countDocuments({
      user_id: req.user.id,
      is_read: false
    });

    res.json({ unreadCount });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages the user alert system by providing efficient endpoints to track and update the status of personalized notifications. It retrieves a chronological list of alerts using .populate() to include sender details and provides a dedicated counter for unread messages to power real-time UI indicators. By allowing individual notifications to be toggled as "read" via their unique ID, the system ensures that users can accurately manage their engagement and clear their activity queues.

- **Organization Controller**

```
import CBOOfficer from "../models/CboOfficerModel.js";
import CBO from "../models/CboModel.js";
import User from "../models/UserModel.js";

// ---------------- CREATE ORGANIZATION ----------------
export const createOrganization = async (req, res) => {
  try {
    const user = await User.findById(req.user.id);

    if (!user) {
      return res.status(404).json({ message: "User not found" });
```

```
    }

    // 🔐 ONLY PRESIDENT CAN CREATE ORG
    const officerRecord = await CBOOfficer.findOne({
      user_id: user._id,
      officer_role: "president",
    });

    if (!officerRecord && user.user_type !== "admin") {
      return res.status(403).json({
            message: "Only organization president or admin can create
organizations",
      });
    }

     const { org_name, acronym, description, logo_url, department } =
req.body;

    const org = await CBO.create({
      org_name,
      acronym,
      description,
      logo_url,
      department,
      created_by: user._id,
      members: [
        {
          user_id: user._id,
          role: "president",
          status: "approved",
        },
      ],
    });

    // 🔥 ALSO CREATE OFFICER ENTRY FOR PRESIDENT
    await CBOOfficer.create({
      user_id: user._id,
      org_id: org._id,
      officer_role: "president",
      display_name: user.display_name,
    });

    res.status(201).json(org);
  } catch (error) {
```

```
    res.status(500).json({ error: error.message });
  }
};

// ---------------- GET ALL ORGANIZATIONS (WITH FILTER FIX) ----------------
export const getOrganizations = async (req, res) => {
  try {
    const { department, search } = req.query;

    let query = {};

    if (department && department !== "all") {
      query.department = department;
    }

    if (search) {
      query.org_name = { $regex: search, $options: "i" };
    }

    const orgs = await CBO.find(query)
      .populate("created_by", "username display_name")
      .populate("members.user_id", "username display_name");

    res.json(orgs);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- GET SINGLE ORGANIZATION ----------------
export const getOrganizationById = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id)
      .populate("created_by", "username display_name")
      .populate("members.user_id", "username display_name");

    if (!org) {
       return res.status(404).json({ message: "Organization not found" });
    }

    res.json(org);
  } catch (error) {
```

```
    res.status(500).json({ error: error.message });
  }
};

// ---------------- JOIN ORGANIZATION (SAFE FIXED) ----------------
export const joinOrganization = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id);

    if (!org) {
        return res.status(404).json({ message: "Organization not found"
});
    }

    const existing = org.members.find(
      m => m.user_id.toString() === req.user.id
    );

    if (existing) {
      return res.status(400).json({
         message: `Already ${existing.status === "pending" ? "requested"
: "a member"}`
      });
    }

    org.members.push({
      user_id: req.user.id,
      role: "member",
      status: "pending"
    });

    await org.save();

    res.json({ message: "Join request sent" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};


// ---------------- APPROVE MEMBER (SECURITY FIXED) ----------------
export const approveMember = async (req, res) => {
  try {
    const { orgId, userId } = req.params;
```

```
    // 🔐 check if requester is officer of org
    const requesterRole = await CBOOfficer.findOne({
      user_id: req.user.id,
      org_id: orgId,
    });

    if (!requesterRole || requesterRole.officer_role === "member") {
      return res.status(403).json({ message: "Not authorized" });
    }

    const org = await CBO.findById(orgId);

     if (!org) return res.status(404).json({ message: "Organization not
found" });

    const member = org.members.find(
      (m) => m.user_id.toString() === userId
    );

    if (!member) {
      return res.status(404).json({ message: "Member not found" });
    }

    // prevent double approval bug
    if (member.status === "approved") {
      return res.status(400).json({ message: "Already approved" });
    }

    member.status = "approved";

    await User.findByIdAndUpdate(userId, {
      organization: orgId,
    });

    await org.save();

    res.json({ message: "Member approved" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- LEAVE ORGANIZATION (SAFE FIX) ----------------
```

```
export const leaveOrganization = async (req, res) => {
  try {
    const org = await CBO.findById(req.params.id);

    if (!org) {
        return res.status(404).json({ message: "Organization not found"
});
    }

    org.members = org.members.filter(
      m => m.user_id.toString() !== req.user.id
    );

    await User.findByIdAndUpdate(req.user.id, {
      organization: null
    });

    await org.save();

    res.json({ message: "Left organization" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller manages the organizational lifecycle and membership within the platform, enforcing strict role-based access control for administrative actions. It allows users to browse and filter organizations by department or name, while ensuring that only authorized "presidents" or admins can create new entities. Additionally, it handles the interactive membership flow—from join requests and officer approvals to leaving an organization—automatically synchronizing the user's status across both the organization and user database records to maintain data integrity.

- **PostController**

```
import Post from "../models/PostModel.js";
import Notification from "../models/NotificationModel.js";
import User from "../models/UserModel.js"; //
import axios from "axios";
import cloudinary from "../config/cloudinary.js";

// ---------------- CREATE POST ----------------
export const createPost = async (req, res) => {
  try {
    const {
      content,
      design_template,
```

```
      spotify_track_url,
      is_anonymous,
      org_id,
      media_url,    // ← change "media" to "media_url"
    } = req.body;

    const post = await Post.create({
      user_id:          req.user.id,
      content:          content || '',
      design_template,
      spotify_track_url,
      is_anonymous:     is_anonymous || false,
      org_id:           org_id || null,
      media: {
        url:  media_url || null,
        type: media_url
                          ?   (media_url.match(/\.(mp4|mov|webm)/i)   ||
media_url.includes('/video/') ? 'video' : 'image')
          : null
      }
    });

    res.status(201).json(post);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- GET POSTS ----------------
export const getPosts = async (req, res) => {
  try {
    const { org, author } = req.query;

    let query = {};
    if (org)    query.org_id  = org;
    if (author) query.user_id = author;

    const posts = await Post.find(query)
      .populate("user_id", "username display_name avatar_url")
      .populate("org_id", "org_name acronym")
      .populate("comments.user_id", "username display_name avatar_url")
      .populate({
        path: "shared_from",
        populate: {
```

```
            path: "user_id",
            select: "username display_name avatar_url"
        }
    })
      .sort({ createdAt: -1 });

    // Transform so frontend gets likes_count + liked_by_me
    const userId = req.user?.id;
    const transformed = posts.map(p => ({
      ...p.toObject(),
      likes_count:  p.reactions.length,
                  liked_by_me:      userId   ?   p.reactions.map(r   =>
r.toString()).includes(userId) : false,
      comments_count: p.comments.length,
    }));

    res.json(transformed);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
// ---------------- SPOTIFY SEARCH ----------------
export const searchSpotify = async (req, res) => {
  try {
    const query = req.query.q;
       if  (!query)  return  res.status(400).json({  message:  "Query  is
required" });

    const tokenRes = await axios.post(
      "https://accounts.spotify.com/api/token",
      new URLSearchParams({ grant_type: "client_credentials" }),
      {
        headers: {
          Authorization: "Basic " + Buffer.from(
                              process.env.SPOTIFY_CLIENT_ID   +   ":"   +
process.env.SPOTIFY_CLIENT_SECRET
          ).toString("base64"),
          "Content-Type": "application/x-www-form-urlencoded",
        },
      }
    );

    const token = tokenRes.data.access_token;
```

```
                                const        response        =        await
axios.get("https://api.spotify.com/v1/search", {
      headers: { Authorization: `Bearer ${token}` },
      params: { q: query, type: "track", limit: 10 },
    });

    res.json(response.data.tracks.items);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- REACT ----------------
export const reactToPost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    const userId = req.user.id;
    const ownerId = post.user_id.toString();

    if (post.reactions.includes(userId)) {
      // Un-react — just remove, no notification
        post.reactions  =  post.reactions.filter(id => id.toString()  !==
userId);
    } else {
      post.reactions.push(userId);

      if (ownerId !== userId) {
        // ← FETCH sender from DB
         const sender = await User.findById(userId).select("display_name
username");

        await Notification.create({
          user_id: ownerId,
          from_user: userId,
          type: "reaction",
            message: `${sender.display_name || sender.username} reacted
to your post`,
          related_id: post._id,
        });
      }
    }

    await post.save();
    res.json({ reactions: post.reactions.length });
```

```
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- COMMENT ----------------
export const addComment = async (req, res) => {
  try {
    const { text } = req.body;
    const post = await Post.findById(req.params.id);

    post.comments.push({ user_id: req.user.id, text });

    if (post.user_id.toString() !== req.user.id) {
      // ← FETCH sender from DB
                                    const      sender      =      await
User.findById(req.user.id).select("display_name username");

      await Notification.create({
        user_id: post.user_id,
        from_user: req.user.id,
        type: "comment",
          message: `${sender.display_name || sender.username} commented
on your post`,
        related_id: post._id,
      });
    }

    await post.save();
    res.json(post.comments);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- SHARE ----------------
export const sharePost = async (req, res) => {
  try {
    const postToShare = await Post.findById(req.params.id);
    const { caption } = req.body;

    // ← If this post is already a share, point to the ORIGINAL post
    const originalId = postToShare.shared_from
      ? postToShare.shared_from
```

```
      : postToShare._id;

    // Fetch the actual original post for the content/spotify
    const originalPost = await Post.findById(originalId);

    const shared = await Post.create({
      user_id:           req.user.id,
      content:           caption || '',
      design_template:   originalPost.design_template,
      spotify_track_url: originalPost.spotify_track_url,
       shared_from:         originalId, // ← always points to the root
original
    });

    // Notify the ORIGINAL post owner, not the sharer
    if (originalPost.user_id.toString() !== req.user.id) {
                                    const      sender      =      await
User.findById(req.user.id).select("display_name username");
      await Notification.create({
        user_id:    originalPost.user_id,
        from_user:  req.user.id,
        type:       "share",
          message:     `${sender.display_name || sender.username} shared
your post`,
        related_id: shared._id,
      });
    }

    res.status(201).json(shared);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- UPDATE ----------------
export const updatePost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.user_id.toString() !== req.user.id)
      return res.status(403).json({ message: "Unauthorized" });

    Object.assign(post, req.body);
    await post.save();
    res.json(post);
```

```
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

// ---------------- DELETE ----------------
export const deletePost = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id);
    if (post.user_id.toString() !== req.user.id)
      return res.status(403).json({ message: "Unauthorized" });

    await post.deleteOne();
    res.json({ message: "Deleted" });
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};

export const getPostById = async (req, res) => {
  try {
    const post = await Post.findById(req.params.id)
      .populate("user_id", "username display_name avatar_url")
      .populate("comments.user_id", "username display_name avatar_url")
      .populate({
        path: "shared_from",
        populate: {
          path: "user_id",
          select: "username display_name avatar_url"
        }
      });

    if (!post) return res.status(404).json({ message: "Post not found" });
    res.json(post);
  } catch (error) {
    res.status(500).json({ error: error.message });
  }
};
```

This controller serves as the dynamic social engine of the platform, managing a complex web of interactions including content creation, Spotify integration, and recursive sharing logic. It handles the full lifecycle of a post from processing media types and authenticated Spotify track searches to generating real-time notifications for reactions, comments, and shares. By transforming raw database records into feature-rich objects with

like counts and "liked-by-me" states, it ensures the frontend can render an interactive and personalized social feed.

- **Profile Controller**

```
import User from "../models/UserModel.js";
import Post from "../models/PostModel.js";
import mongoose from "mongoose";

// GET LOGGED-IN USER PROFILE
export const getProfile = async (req, res) => {
  try {
    const user = await User.findById(req.user.id)
      .select("-password")
      .populate("organization", "org_name acronym")
      .populate("department", "name");

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

     const postsCount = await Post.countDocuments({ user_id: user._id
});

    res.json({
      user,
      postsCount
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// UPDATE LOGGED-IN USER PROFILE
export const updateProfile = async (req, res) => {
  try {
    const { display_name, bio, avatar_url } = req.body;

    const updatedUser = await User.findByIdAndUpdate(
      req.user.id,
      {
        display_name,
        bio,
        avatar_url
      },
      { returnDocument: "after" }
```

```
    )
      .select("-password")
      .populate("organization", "org_name acronym")
      .populate("department", "name");

    if (!updatedUser) {
      return res.status(404).json({ message: "User not found" });
    }

    res.json({
      message: "Profile updated successfully",
      user: updatedUser
    });
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

// GET OTHER USER PROFILE
export const getUserProfileById = async (req, res) => {
  try {
    const { id } = req.params;

    // VALIDATE OBJECT ID
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid user ID" });
    }

    const user = await User.findById(id)
      .select("-password")
      .populate("organization", "org_name acronym")
      .populate("department", "name");

    if (!user) {
      return res.status(404).json({ message: "User not found" });
    }

     const postsCount = await Post.countDocuments({ user_id: user._id
});

    res.json({
      user,
      postsCount
    });
```

```
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};

//  GET POSTS OF SPECIFIC USER
export const getUserPosts = async (req, res) => {
  try {
    const { id } = req.params;

    // VALIDATE OBJECT ID
    if (!mongoose.Types.ObjectId.isValid(id)) {
      return res.status(400).json({ message: "Invalid user ID" });
    }

    const posts = await Post.find({ user_id: id })
      .populate("user_id", "username display_name avatar_url")
      .populate("org_id", "org_name acronym")
      .populate("comments.user_id", "username display_name avatar_url")
      .sort({ createdAt: -1 });

    const transformedPosts = posts.map(post => ({
      ...post.toObject(),
      likes_count: post.reactions.length,
      comments_count: post.comments.length
    }));

    res.json(transformedPosts);
  } catch (error) {
    res.status(500).json({ message: error.message });
  }
};
```

This controller manages user identity and personal data visibility by providing secure methods to retrieve and modify profile information. It aggregates data from multiple collections—excluding sensitive fields like passwords—to return a complete user profile that includes organizational affiliations, department details, and a dynamic count of the user's total posts. Furthermore, it handles public-facing profiles and user-specific feeds, ensuring that ID validation is performed before fetching and transforming post data into a frontend-ready format with calculated reaction and comment totals.

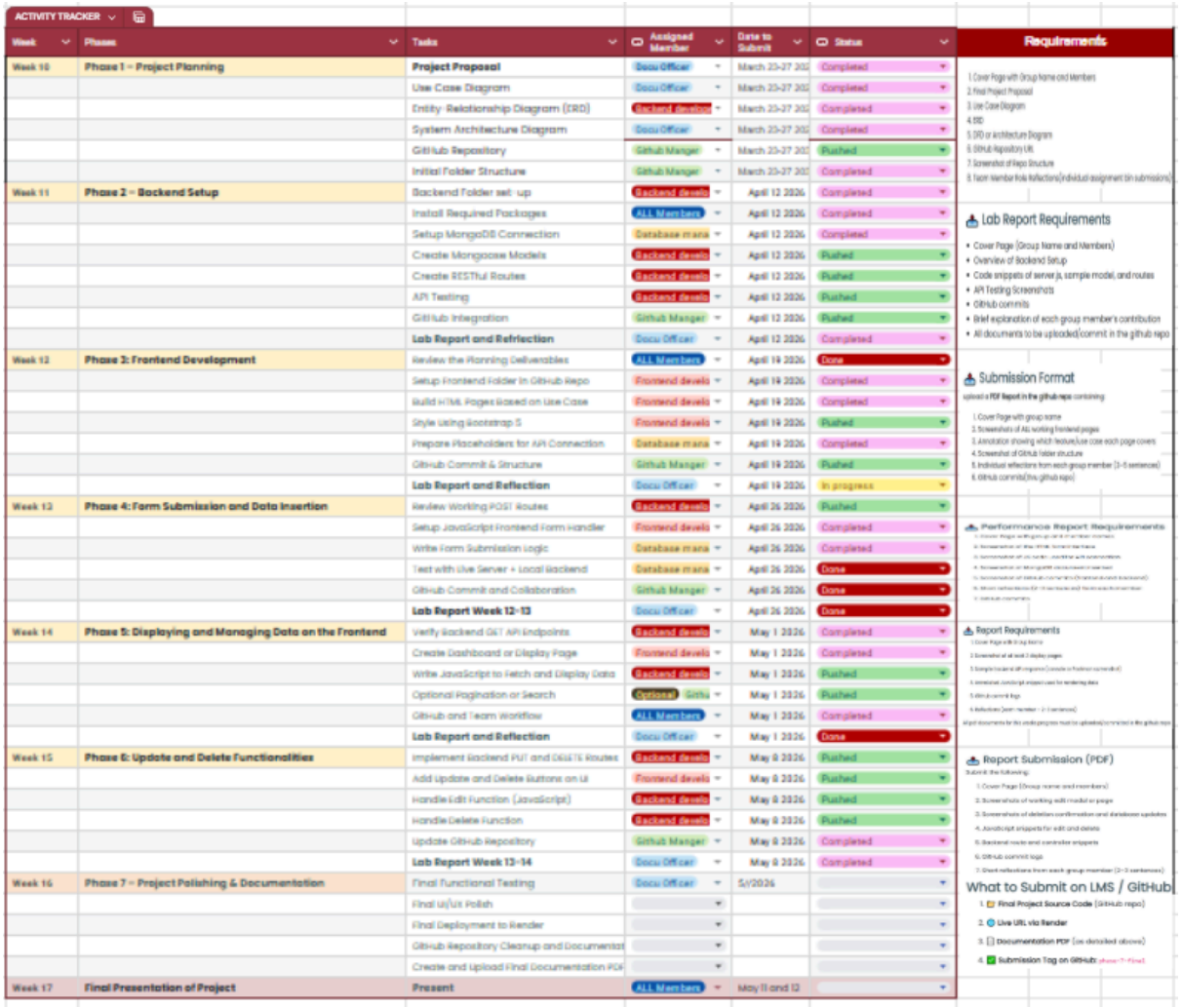
- **Upload Controller**

```
import cloudinary from "../config/cloudinary.js";

export const uploadAvatar = async (req, res) => {
  try {
    const { image } = req.body;

    const result = await cloudinary.uploader.upload(image, {
      folder: "dearbup/profile",
    });

    res.json({
      imageUrl: result.secure_url,
    });
  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
};

export const uploadPostMedia = async (req, res) => {
  try {
    const { file, mediaType } = req.body;

    if (!file) {
      return res.status(400).json({
        message: "No media file provided",
      });
    }

    const detectedType = mediaType || (
      file.startsWith("data:video") ? "video" : "image"
    );

    const result = await cloudinary.uploader.upload(file, {
      folder: "dearbup/posts",
      resource_type: detectedType === "video" ? "video" : "image",
    });

    res.json({
      mediaUrl: result.secure_url,
      mediaType: detectedType,
    });
```

```
  } catch (error) {
    res.status(500).json({
      message: error.message,
    });
  }
};
```

This controller facilitates cloud-based asset management by interfacing with Cloudinary to store and serve user-generated media. It processes incoming data URIs for both profile avatars and post attachments, automatically detecting whether the content is an image or video to apply the correct storage constraints and folder organization. Once the upload is successful, it returns a secure URL to the frontend, allowing the application to display high-quality media without taxing the local server's storage or bandwidth.

TRACKER UPDATE



FOOTNOTE:

There is a t least 10% of AI used in this lab report because some terms are not easy to explain and there are complex words and a code snippets that are not easy to explain.

REFLECTIONS

Ivan Jacob Milan | Backend Developer

For Week 15, I focused on backend development by implementing PUT and DELETE routes to handle updating and deleting data. I tested these endpoints to ensure that changes were correctly reflected in the database. I also integrated Cloudinary to support uploading photos and videos in posts, which added more functionality to the system. In addition, I created a backend feature for profile viewing so users can access their information. While working on these tasks, I encountered some issues with data handling and validation, but I was able to resolve them through debugging and careful testing.

James P. Espinosa | Document Officer | Project Manager

As a document officer, I searched for every part of code to input snippets on this week's lab report. My role begins when I prepare the screenshots needed and explain every part of the display and even document some of the js. Moreover, I updated all the tasks in the activity tracker and prioritized completing the remaining requirements.

Erick John B. Bonaobra | Frontend Developer

As the frontend developer on this group, on week 15 I focused on creating the edit and delete modal for the post. I also fixed the notification, if someone shared your post and deleted it you can see a message pop up on the bottom right side saying that the post is deleted. I also apply the delete and edit button to the profile tab.

Manuel Angelo A. Loma | GitHub Manager

This week, I contributed to our group project by helping the Frontend Manager with different tasks. One of my main responsibilities was helping test and debug the code to make sure the system worked properly and errors were fixed immediately. I assisted in checking different features, identifying bugs, and helping improve the overall functionality of the project. At the same time, I continued managing our GitHub repository by organizing files, checking updates, and handling branches. This helped our team work more smoothly and avoid conflicts in our code. Even though we faced some issues such as merge conflicts and syncing problems, we were able to solve them through teamwork and good communication. Overall, this week helped me improve my debugging and frontend skills, gain more experience in using GitHub, and contribute more effectively to the progress of our project.

Nadel B. Volante | Database Manager

As the Database Manager for Phase 6, I ensured that the update and delete functions properly affected the MongoDB database. I tested the PUT and DELETE endpoints, verified that records were updated or removed correctly, and checked that frontend requests matched the correct database documents. This phase improved my

understanding of how frontend, backend, and database systems work together in CRUD operations.